ITS 365 – Project Phase 1 – CIFAR-10

By Marcus Nunez and Hakeem Atanda

I. Project Goals & Input

The goal of this project is to analyze and classify the CIFAR dataset. This dataset was imported and extracted via ‘zipfile’, before being split between training and verification.

```
# extract dataset and load

zip_file_path = '/content/drive/MyDrive/its365/CIFAR-10-images-master.zip'
extracted_dir = '/mnt/data/extracted_cifar10'

with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extracted_dir)

dataset_dir = os.path.join(extracted_dir, 'CIFAR-10-images-master')
data_dir = os.path.join(dataset_dir, 'train')
full_dataset = torchvision.datasets.ImageFolder(os.path.join(dataset_dir, 'train'), transform=transform)
```

II. Model Classification

This model is heavily based on my previous SmileCNN, which classified whether images contained a smile or not. This utilizes a 3-layer CNN at 80% training and 20% validation. Before application, data is ran through filters of various image sizes (32, 64,128), as well as ReLU and training dropout checks.

```
# define CNN model
class ProjectCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.Dropout2d(0.1),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Dropout2d(0.2),
            nn.MaxPool2d(2),

            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.Dropout2d(0.3),
            nn.MaxPool2d(2)
        )

        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Sequential(
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 10)  # CIFAR-10 has 10 classes
        )

    def forward(self, x):
        x = self.conv(x)
        x = self.pool(x)
        x = x.view(x.size(0), -1)  # Flatten the output of convolutional layers
        return self.fc(x)
```

```
# split 80/20
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size])

# create dataloaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)

# handle weight imbalance
class_count = np.bincount([y for _, y in train_dataset])
weights = 1. / class_count  # Inverse of class frequency
sample_weights = np.array([weights[y] for _, y in train_dataset])
sampler = WeightedRandomSampler(sample_weights, len(sample_weights))

# update train with new handle
train_loader = DataLoader(train_dataset, batch_size=64, sampler=sampler)
```

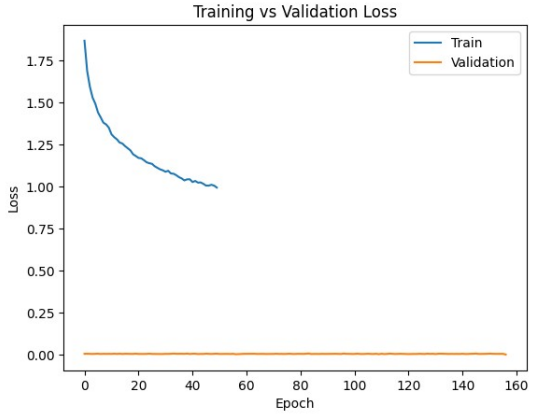
III. Training Method

This model trains with 50 epochs, attempting to lower the loss, each training loop ending when it it detects loss variation is less than previously.

```
# training loop
num_epochs = 50
train_losses = []
val_losses = []

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

    train_loss = running_loss / len(train_loader.dataset)
    train_losses.append(train_loss)
```

IV. Results

Utilizing CoLab's T4 GPU, this model was able to process in ~40 minutes, yielding favorable results. According to the graph generated through matplotlib, training was relatively effective throughout the training, loss correction gradually receding, but maintaining effectiveness.



Classification and predictions both performed relatively the same, both achieving an F1 score of ~0.72 and an accuracy of 72%. However, each model seems to struggle with different categories, indicating that the model has a lot of cases of overlapping classes.

Validation Set Evaluation:

	precision	recall	f1-score	support
0	0.76	0.74	0.75	996
1	0.87	0.84	0.85	1013
2	0.65	0.58	0.61	954
3	0.52	0.42	0.47	983
4	0.66	0.67	0.66	997
5	0.59	0.72	0.65	1020
6	0.73	0.79	0.76	1025
7	0.77	0.76	0.76	975
8	0.84	0.82	0.83	1020
9	0.80	0.83	0.82	1017
accuracy			0.72	10000
macro avg	0.72	0.72	0.72	10000
weighted avg	0.72	0.72	0.72	10000

Validation Confusion Matrix:

```
[[733  14  62  20  11  12   6  21  81  36]
 [  8 846   4   9   5   1  12   3  22 103]
 [ 68   2 558  47  93  62  78  36   5   5]
 [ 25   3  53 416  59 276  95  28  12  16]
 [ 24   1  65  38 671  42  59  87   8   2]
 [  3   1  35 131  37 737  27  36   3  10]
 [  1   2  35  67  65  33 811   3   2   6]
 [  9   2  30  28  69  86   6 737   2   6]
 [ 63  31  17  20  11   3  10   3 835  27]
 [ 28  69   4  26   3   5   7   6  23 846]]
```

Test Set Evaluation:

	precision	recall	f1-score	support
0	0.75	0.76	0.76	1000
1	0.88	0.84	0.86	1000
2	0.64	0.56	0.59	1000
3	0.54	0.45	0.49	1000
4	0.65	0.69	0.66	1000
5	0.58	0.70	0.64	1000
6	0.72	0.80	0.75	1000
7	0.77	0.74	0.76	1000
8	0.87	0.82	0.84	1000
9	0.80	0.83	0.81	1000
accuracy			0.72	10000
macro avg	0.72	0.72	0.72	10000
weighted avg	0.72	0.72	0.72	10000

Test Confusion Matrix:

```
[[764  14  65  22   8   5  11  11  60  40]
 [ 17 841   3   5   4   4  14   5  15  92]
 [ 66   3 555  48 102  81  92  37   9   7]
 [ 15   2  66 454  62 254  93  25  10  19]
 [ 28   3  54  44 685  42  59  79   6   0]
 [  7   4  45 127  51 705  17  40   1   3]
 [  3   3  31  71  59  24 796   5   6   2]
 [ 13   0  32  39  81  76   4 744   1  10]
 [ 85  18   7  11   2   7  13   1 822  34]
 [ 22  63   8  17   8   8  10  19  19 826]]
```

The confusion matrix visually details this test model's confusion, most of which seems to be between dogs and cats (classes 3 and 5), as well as birds being confused for deer/frogs (classes 2 and 4/6). Overall model improvement can be focused towards these categories, testing and optimization continuing until all class confusions are lowered.